

LangSmith vs Langfuse

Same agent. Same four test questions. Same `gpt-4o` judge. The only thing that changes is which tool is watching.

Every screenshot below is from this lab. Every number is measured, not quoted from a docs page.

- **LangSmith**: hosted, at `smith.langchain.com`
- **Langfuse**: self-hosted, six Docker containers on `localhost:3000`, `v4.6.0` OSS

What we are watching

A small LangGraph agent with three tools:

- `calculator` , always works
- `get_weather` , fake data that **fails 20% of the time on purpose**
- `search_docs` , tiny RAG over five markdown files

The 20% failure is the interesting part. It is what makes the error screens below real instead of staged.

Contents

Document	Covers
01-tracing.md	Turning tracing on, reading a trace, error handling, grouping runs
02-evaluation.md	Golden datasets, LLM-as-judge, measured scores
03-features.md	Dashboards, and what each tool has that the other does not
04-cost.md	Pricing for both, and measured CPU/RAM/disk for self-hosted Langfuse
05-scale.md	How many traces self-hosted Langfuse handles, load tested, versus LangSmith's limits

PDF versions of all of the above are in [pdf/](#), including a single combined [langfuse-vs-langsmith-FULL.pdf](#) with everything in one file. Screenshots are embedded, so the PDFs are self-contained and safe to send on.

Running this yourself

```
# plain app, traced by LangSmith automatically
uv run python -m src.agent "what's the weather in Chennai and what's 100/7?"

# same agent, explicitly traced into Langfuse
uv run python -m src.obs_langfuse.traced_run "what's the weather in Chennai and what's 100/7?"

# datasets (run once each)
uv run python -m src.obs_langsmith.dataset
uv run python -m src.obs_langfuse.dataset

# evaluations
uv run python -m src.obs_langsmith.evaluate
uv run python -m src.obs_langfuse.evaluate
```

To force a failure instead of waiting for the 20% dice:

```
PYTHONPATH=. uv run python -c "
import random
random.random = lambda: 0.0
from dotenv import load_dotenv; load_dotenv()
from src.agent.graph import graph
graph.invoke({'messages': [('user', 'what is the weather in Chennai?')]}, config={'run_name': 'for
"
```

Verdict

What you care about	Pick
Tracing working in ten seconds	LangSmith
Data must not leave your network	Langfuse
Seeing <i>where</i> an agent broke	Langfuse
Judge explaining its scores	Langfuse
Nothing to operate	LangSmith
Filtering on your own metadata keys	LangSmith

There is no overall winner. If your data can leave the building, LangSmith gets you further in an afternoon and there is nothing to maintain. If it cannot, Langfuse is a real replacement rather than a downgrade, and its error views are genuinely better. The price is six containers and a port conflict.

Tracing and debugging

How each tool is switched on, what a run looks like, and what happens when the agent breaks.

Turning tracing on

This is the biggest difference, and it is visible in the code.

LangSmith: one environment variable

```
LANGSMITH_TRACING=true
```

That is it. No imports, no code changes, nothing passed into the agent.

Proof is in the run list. The rows named `LangGraph` came from running the plain app:

```
uv run python -m src.agent "what's the weather in Chennai and what's 100/7?"
```

That file does not mention LangSmith anywhere. It still shows up with full latency, token and cost data.

Personal / Tracing / ilm-observability-lab

ilm-observability-lab ID Tracing Evaluators Automations Insights Engine New

Default View Last 1 day Threads Traces Runs Input Search input content

Add filter <>

2

nts 1

Beta

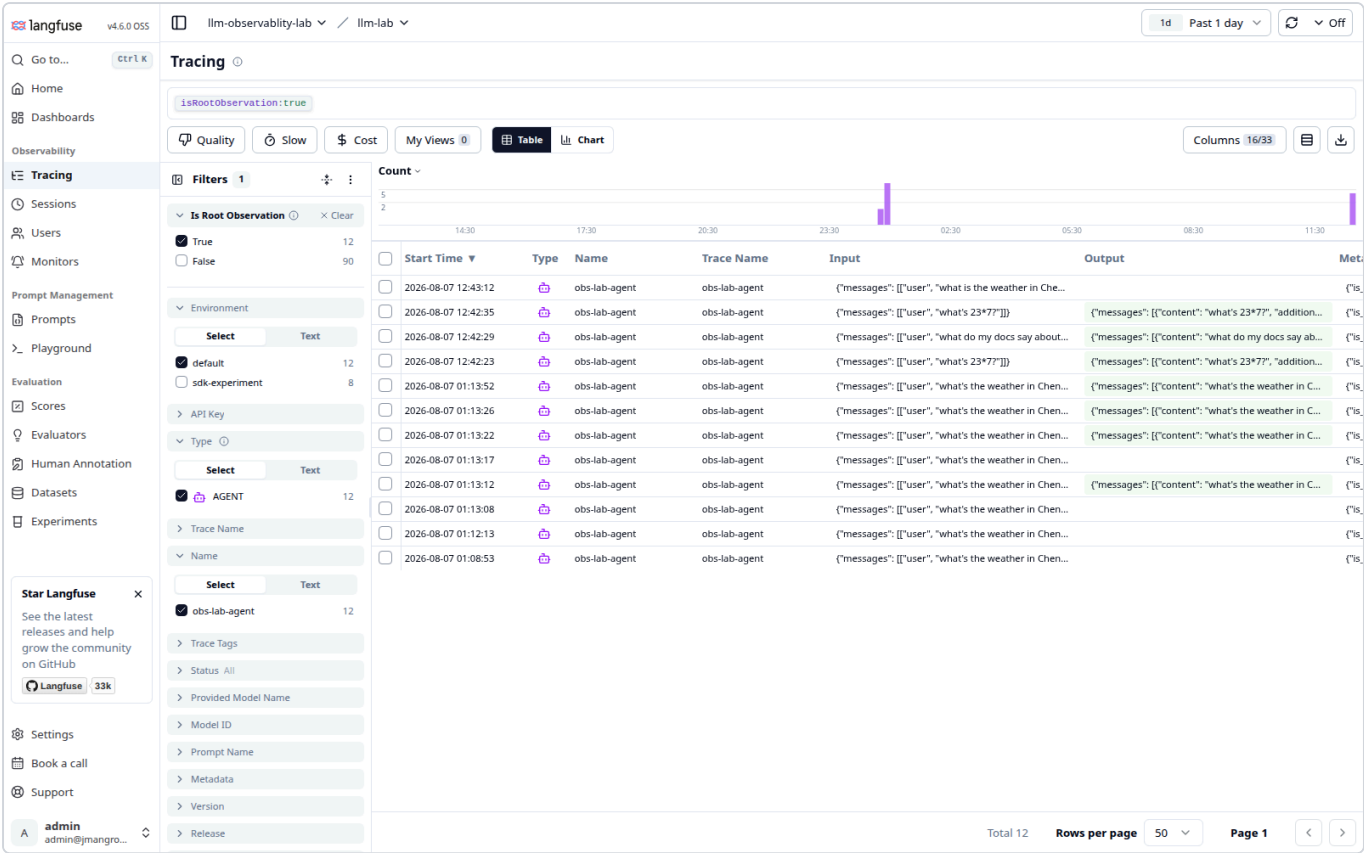
☐	☒	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation ...	Tokens	Cost	First Token	Tags
☐	☒	forced-weather-failure	user: what is the weather ...		RuntimeError("weather s...	8/7/2026, 11:51...	1.36s			279	\$0.0008175		
☐	☒	LangGraph	user: what's the weather ...	ai: The weather in Chenn...		8/7/2026, 11:4...	2.13s			690	\$0.002295		
☐	☒	obs-lab-tagged-run	user: what's the weather ...	ai: The weather in Chenn...		8/7/2026, 11:41...	2.77s			690	\$0.002295		phase3
☐	☒	LangGraph	user: what do my docs s...	ai: Your docs say to retry ...		8/7/2026, 11:35...	3.96s			878	\$0.002795		
☐	☒	obs-lab-tagged-run	user: what's the weather ...	ai: The weather in Chenn...		8/7/2026, 11:3...	2.18s			690	\$0.002295		manual-te
☐	☒	ChatOpenAI	human: You are grading ...	ai: CORRECT The agent's...		8/7/2026, 11:17...	0.54s			130	\$0.0004675		
☐	☒	LangGraph	user: what's 23*7?	ai: 23 * 7 equals 161.		8/7/2026, 11:17...	0.99s			577	\$0.001645		
☐	☒	LangGraph	user: what's the weather ...		RuntimeError("weather s...	8/7/2026, 11:17...	0.63s			317	\$0.0011525		
☐	☒	ChatOpenAI	human: You are grading ...	ai: CORRECT The agent's...		8/7/2026, 11:17...	0.62s			236	\$0.0008225		
☐	☒	LangGraph	user: what do my docs s...	ai: Your docs say to retry ...		8/7/2026, 11:17...	3.26s			878	\$0.002795		
☐	☒	LangGraph	user: weather in Mumbai,...		RuntimeError("weather s...	8/7/2026, 11:17...	1.41s			282	\$0.0008175		
☐	☒	LangGraph	user: what's the weather ...		RuntimeError("weather s...	8/7/2026, 11:15...	0.66s			317	\$0.0011525		
☐	☒	ChatOpenAI	human: You are grading ...	ai: CORRECT The agent's...		8/7/2026, 11:15...	0.56s			133	\$0.000475		
☐	☒	LangGraph	user: what's 23*7?	ai: The result of 23 * 7 is 1...		8/7/2026, 11:15...	1.11s			581	\$0.001685		
☐	☒	ChatOpenAI	human: You are grading ...	ai: First line: CORRECT Se...		8/7/2026, 11:15...	0.78s			288	\$0.0009975		
☐	☒	LangGraph	user: what do my docs s...	ai: Your docs say: - Retry ...		8/7/2026, 11:15...	14.07s			924	\$0.003255		

Look at the columns you get for free: `Latency` , `Tokens` , `Cost` , `Error` , `Tags` . The red rows are the weather tool failing. Nobody wrote error handling to make those appear.

Langfuse: one handler per call

```
handler = CallbackHandler()
graph.invoke(..., config={"callbacks": [handler]})
```

Nothing is recorded unless you pass that handler.



Which is better depends on what you care about. LangSmith gets you tracing in ten seconds, but it also captures everything in the process, including code you did not write. Langfuse costs you a line at every call site, and in exchange nothing leaves the process unless you asked for it.

One useful side effect: with both sets of keys in `.env`, a single run lands in **both** tools at once. Worth doing once.

What a single run looks like

Both give you the same core thing: a tree of what happened, with time and tokens on each node.

LangSmith

The screenshot displays the LangGraph web interface. On the left, a sidebar shows a list of runs under the 'llm-observability-lab' project. The central 'Trace' panel shows a tree view of the execution steps for a specific run. The root node is 'LangGraph' with a duration of 2.13s and 690 tokens. It branches into an 'agent' node (1.55s) which calls 'ChatOpenAI' (1.26s) and 'should_continue' (0.00s). The 'agent' node then calls 'tools' (0.00s), which includes 'get_weather' and 'calculator'. The 'tools' node then calls another 'agent' node (0.57s), which again calls 'ChatOpenAI' (0.57s) and 'should_continue' (0.00s). The right panel shows the 'Attributes' for the selected step, including metadata like 'revision_id', 'LANGSMITH_ENDPOINT', 'LANGSMITH_PROJECT', and runtime information like 'langchain_core_version' and 'langchain_version'.

2.13s , 690 tokens, \$0.0023 at the root. The graph:step and seq:step labels come from LangGraph and show the execution order.

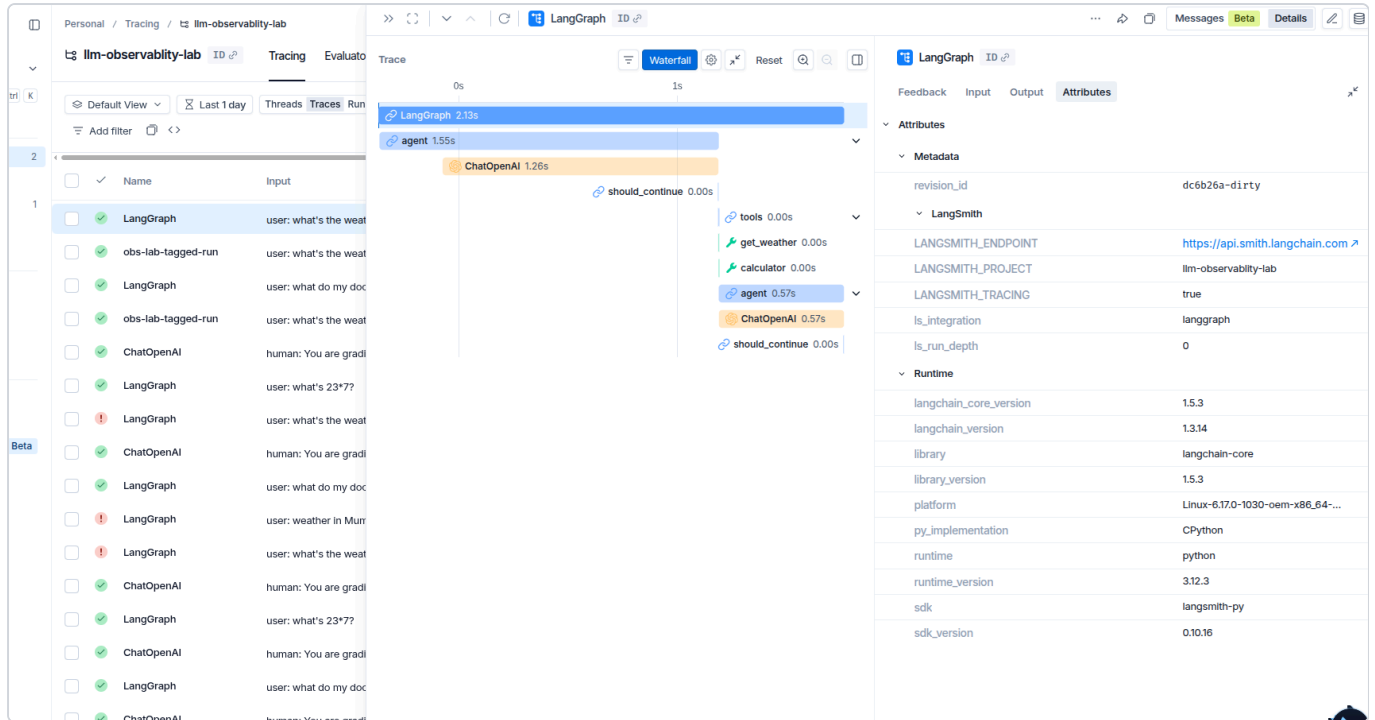
Langfuse

The screenshot displays the Langfuse web interface. On the left, a sidebar shows navigation options like 'Home', 'Dashboards', 'Observability', 'Tracing', 'Sessions', 'Users', 'Monitors', 'Prompt Management', 'Prompts', 'Playground', 'Evaluation', 'Scores', 'Evaluators', 'Human Annotation', 'Datasets', and 'Experiments'. The central 'Trace' panel shows a tree view of the execution steps for a specific run. The root node is 'obs-lab-agent' with a duration of 1.87s and \$0.001645. It branches into an 'agent' node (1.35s) which calls 'ChatOpenAI' (1.15s) and 'should_continue' (0.00s). The 'agent' node then calls 'tools' (0.00s), which includes 'calculator' (0.00s). The 'tools' node then calls another 'agent' node (0.51s), which again calls 'ChatOpenAI' (0.51s) and 'should_continue' (0.00s). The right panel shows the 'Preview' and 'Metadata' for the selected step. The 'Preview' section includes 'Tags', 'Input', 'Calculator', 'Assistant', and 'Corrected Output'. The 'Metadata' section includes a table of attributes like 'is_langchain_root', 'is_integration', 'scope.attributes.public_key', 'scope.version', 'scope.name', 'resourceAttributes.service.name', 'resourceAttributes.service.instance.id', 'resourceAttributes.telemetry.sdk.version', 'resourceAttributes.telemetry.sdk.name', and 'resourceAttributes.telemetry.sdk.language'.

Same shape, plus a rendered graph of the agent loop underneath the tree.

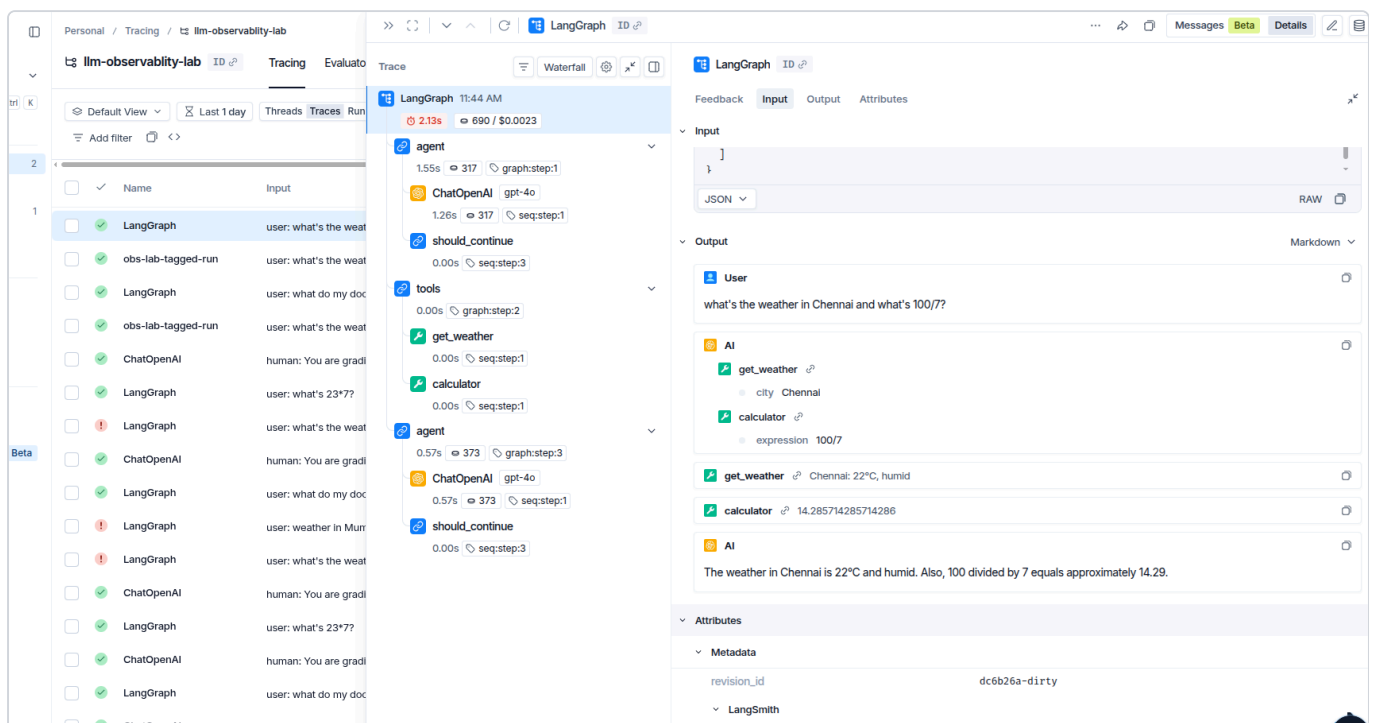
Timing view

LangSmith has a waterfall that makes it obvious where the time actually went:



The whole run is 2.13s and the ChatOpenAI call is 1.26s of it. Both tool calls are 0.00s. The model is the slow part, not your code. That is nearly always the answer, and it is worth seeing once.

The conversation itself

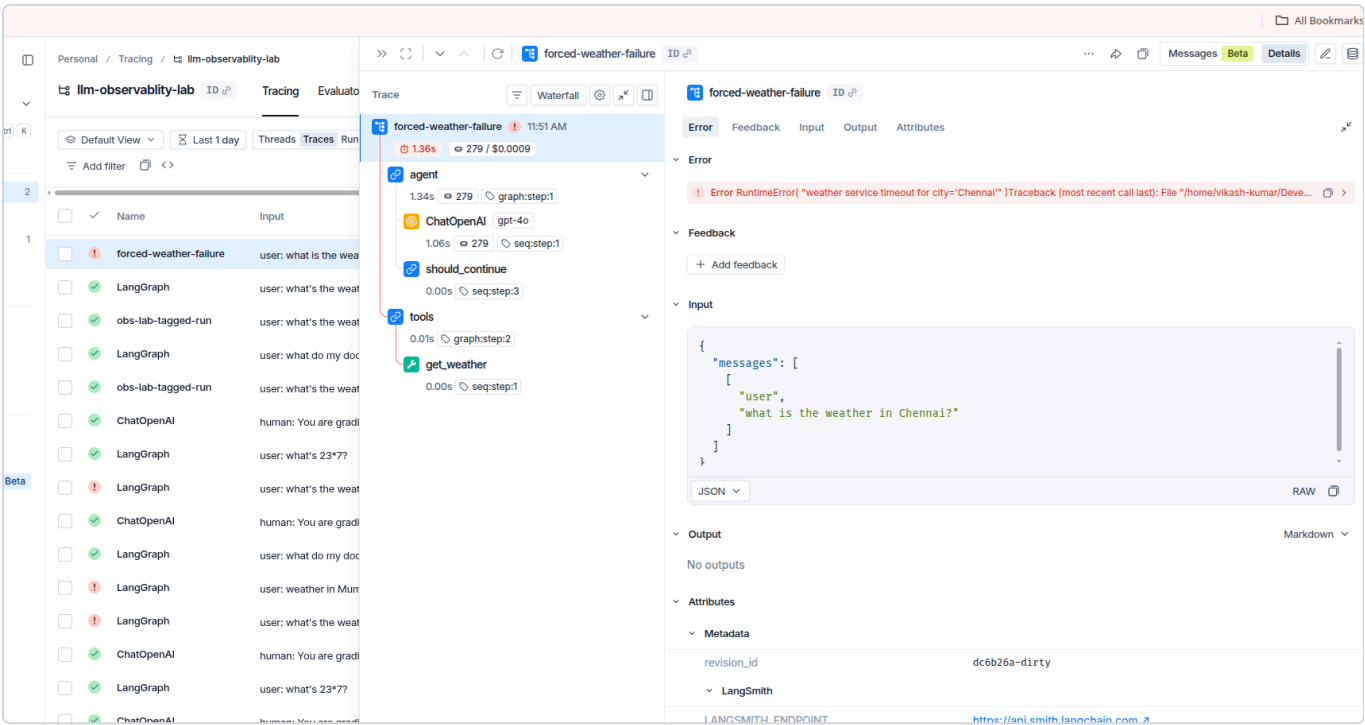


Both tools show the full message flow: what the model asked for, what each tool returned, what came back at the end.

When something breaks

This is where the two tools genuinely differ, and it matters more than any feature list.

LangSmith marks the root of the trace and puts the full Python traceback in a dedicated **Error** tab. Output says **No outputs**.



Langfuse marks every span the error passed through, all the way up.

Read the Langfuse tree carefully:

```
obs-lab-agent  ERROR
  agent
    ChatOpenAI      1.30s  270 -> 48  $0.001155
    should_continue
  tools             ERROR
    get_weather     ERROR  0.00s
    calculator      0.00s
```

`get_weather` failed, so `tools` is marked, so the whole trace is marked. `calculator` sits right next to it, untouched and green. The graph below even paints `__end__` red.

Verdict on errors: Langfuse wins. LangSmith tells you *that* the run failed and hands you a traceback. Langfuse shows you *where* in the agent it failed without you reading a single line of Python.

One detail worth knowing: `get_weather` shows `0.00s`. A real network timeout would take seconds. Zero means it failed on the first line of the function. That is how you tell a fake failure from a real one.

Grouping runs together

Both can thread separate runs into one conversation.

LangSmith calls them Threads:

The screenshot displays the LangSmith interface for a thread titled 'llm-observability-lab'. The left sidebar contains navigation options like 'Experiments', 'Queues', and 'Shortcuts'. The main area shows a list of messages and their corresponding AI responses. The 'INPUTS' section shows a user message: 'what's the weather in Chennai and what's 100/7?'. The 'OUTPUTS' section shows the AI's response, which includes a 'get_weather' tool call and a 'calculator' tool call. The 'MESSAGES' section shows the conversation history. The 'STATS' section on the right provides summary information: 'First start: 08/07/2026, 12:40:22 AM GMT+5', 'Last end: 08/07/2026, 11:41:07 AM GMT+5', 'Latency: P50: 2.0s, P99: 2.0s', and 'Total cost breakdown: Input: 74% (2.3K), Output: 26% (207), Total: 100% (2.5K)'.

Langfuse calls them Sessions:

The screenshot displays the Langfuse interface for a session titled 'llm-observability-lab'. The left sidebar contains navigation options like 'Home', 'Dashboards', 'Observability', 'Sessions', 'Users', 'Monitors', 'Prompt Management', 'Prompts', 'Playground', 'Evaluation', 'Scores', 'Evaluators', 'Human Annotation', 'Datasets', and 'Experiments'. The main area shows a table of sessions. The 'Filters' section on the left allows filtering by 'Environment' (default, sdk-experiment), 'Session ID', 'User IDs', 'Trace Tags', 'Metadata', 'Bookmarked' (Bookmarked, Not bookmarked), 'Session Duration', 'Traces Count', 'Input Tokens', 'Output Tokens', 'Total Tokens', 'Input Cost', 'Output Cost', 'Total Cost', 'Categorical Scores', 'Numeric Scores', 'Boolean Scores', 'Comment Count', and 'Comment Content'. The table shows one session: 'session-001' with 'Created At: 2026-08-07 01:08:53', 'Duration: 11h 34m 20s', 'Environment: default', 'User IDs: vikash', and 'Traces: 12'. The bottom right shows 'Rows per page: 50', 'Page: 1 of 1'.

One session, session-001 , holding 12 traces over 11 hours, attributed to user vikash .

That 11 hour duration is a warning, not a feature. The session ID is hardcoded in [traced_run.py](#), so every run ever executed piles into the same session. In a real app you generate one per conversation.

The two tools name these fields differently:

	LangSmith	Langfuse
Tags	<code>config["tags"]</code>	<code>metadata["langfuse_tags"]</code>
User	<code>metadata["user_id"]</code> , any key you like	<code>metadata["langfuse_user_id"]</code> , reserved name
Session	<code>metadata["session_id"]</code> , any key you like	<code>metadata["langfuse_session_id"]</code> , reserved name

LangSmith lets you invent your own keys and filter on them. Langfuse only promotes specific `langfuse_` - prefixed keys into real UI features, and treats everything else as plain metadata. You trade magic strings for purpose-built screens.

Datasets and evaluation

Running the same four golden questions through both tools with an LLM judge.

Datasets and evaluation

Both hold a golden dataset and run your agent against it with an LLM judge.

The exact same four questions were loaded into both, imported from one Python list, so the comparison is fair.

langfuse

v4.6.0 OSS

llm-observability-lab / llm-lab / Datasets / llm-obs-lab-golden

+ New item

Upload CSV

Go to...

ctrl K

Home

Dashboards

Observability

Tracing

Sessions

Users

Monitors

Prompt Management

Prompts

Playground

Evaluation

Scores

Evaluators

Human Annotation

Datasets

Experiments

Star Langfuse

See the latest releases and help grow the community on GitHub

Langfuse

33k

Settings

Book a call

Support

admin

admin@jmgro...

Dataset llm-obs-lab-golden

Items Experiments

Search...

IDs

Filters

Columns 8/8

Item id	Source	Status	Created At	Input	Expected Output	Metadata	Actions
b6d92353-9d59-4...		Active	2026-08-07 11:15:28	{ "items": { "question": "weather in Mumbai, then multiply the temperature by 2" } }	{ "items": { "answer": "States Mumbai's current temperature from the weather tool and gives that temperature multiplied by 2." } }		
dc09870c-1e5e-4...		Active	2026-08-07 11:15:28	{ "items": { "question": "what do my docs say about retries?" } }	{ "items": { "answer": "Retry only idempotent operations, use exponential backoff with jitter, cap attempts at 3-5, retry on 429/5xx only" } }		
a0e8a271-4021-4...		Active	2026-08-07 11:15:28	{ "items": { "question": "what's the weather in Chennai and what's 100/7?" } }	{ "items": { "answer": "Chennai is 22°C and humid; 100/7 is approximately 14.29." } }		
8c64aa63-eec8-4...		Active	2026-08-07 11:15:28	{ "items": { "question": "what's 23*7?" } }	{ "items": { "answer": "161" } }		

Rows per page 50

Page 1 of 1

Running the experiment gives you per-item results:

langfuse

v4.6.0 OSS

llm-observability-lab

llm-lab

Experiments

Go to...

CTRL K

Experiment

Baseline

baseline-gpt4o - 2026-08-07T05...

Experiment selection

Search experiments...

Display

Results

Analytics

My Views

Columns 9/12

Filters

Item Metadata

Metadata

Scores (categorical)

Scores (numeric)

Scores (boolean)

Trace Scores (categorical)

Trace Scores (numeric)

Trace Scores (boolean)

Input

question: "what's 23*7?"

Expected Output

answer: "161"

baseline-gpt4o - 2026-08-07T05:47:06.202042Z

Baseline

OUTPUT

"23 * 7 equals 161."

SCORES

correctness

1.00

METADATA

Item ID

8c64aa63-eec8-492b-9549-7d8c8b581bbe

Observation

0b85516cf67ac449

Execution Trace

ed09055d5e462fa81a8a7273dc944f7e

Total Cost

\$0.001645

Latency

6.96s

question: "what's the weather in chennai and what's 100/7?"

answer: "Chennai is 22°C and humid; 100/7 is approximately 14.29."

OUTPUT

"Error: weather service timeout for city='Chennai'"

SCORES

correctness

-

METADATA

Item ID

a0e8a271-4021-4c11-a300-f765329a1a3c

Observation

c21cba4bcb3bbf6

Execution Trace

2daea7e1edec112cea63b4586cfea900

Total Cost

\$0.001153

Latency

5.97s

Rows per page

20

Page

1

of 1

<<

<

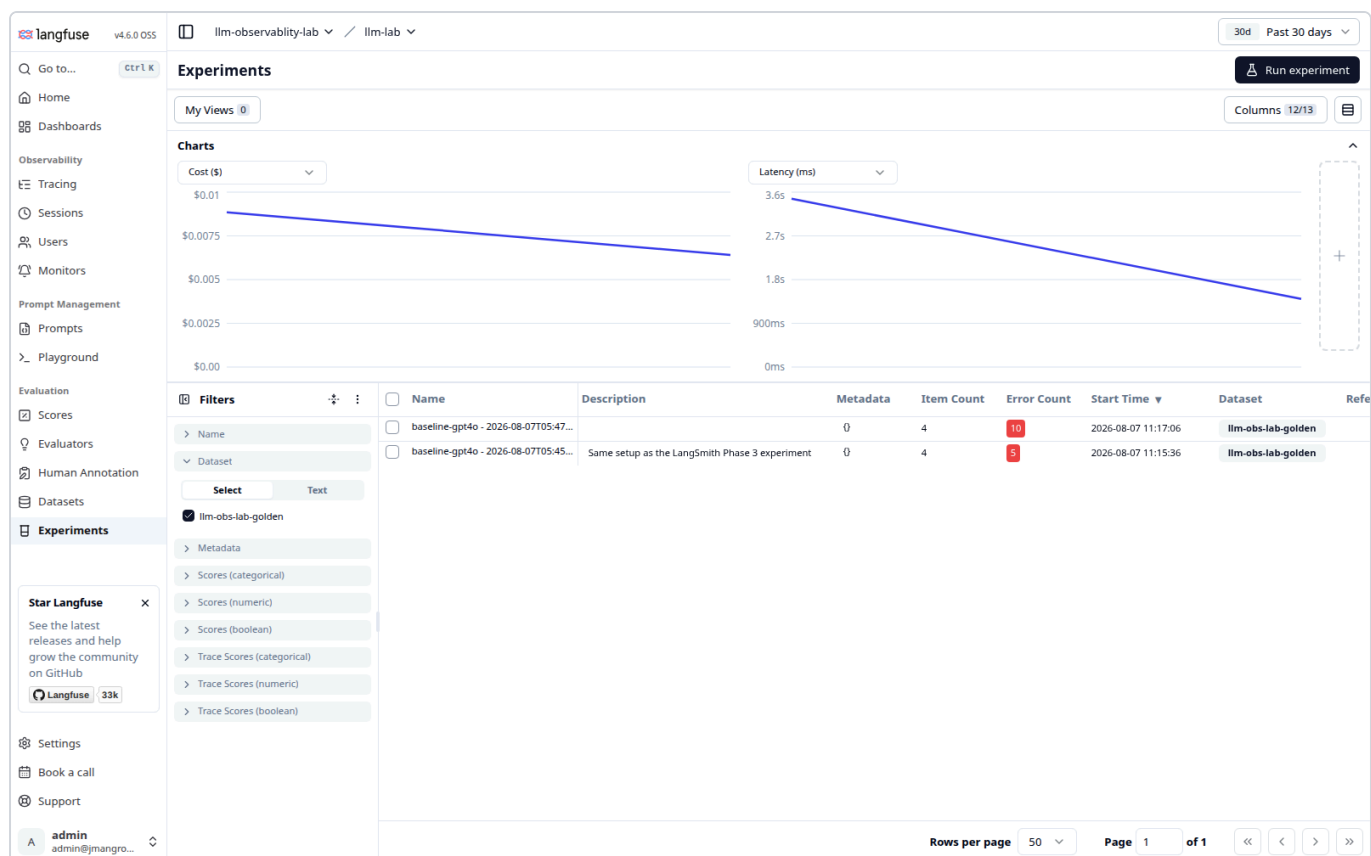
>

>>

Each row shows the question, the expected answer, what the agent actually said, the `correctness` score, cost, latency, and a link straight to the execution trace.

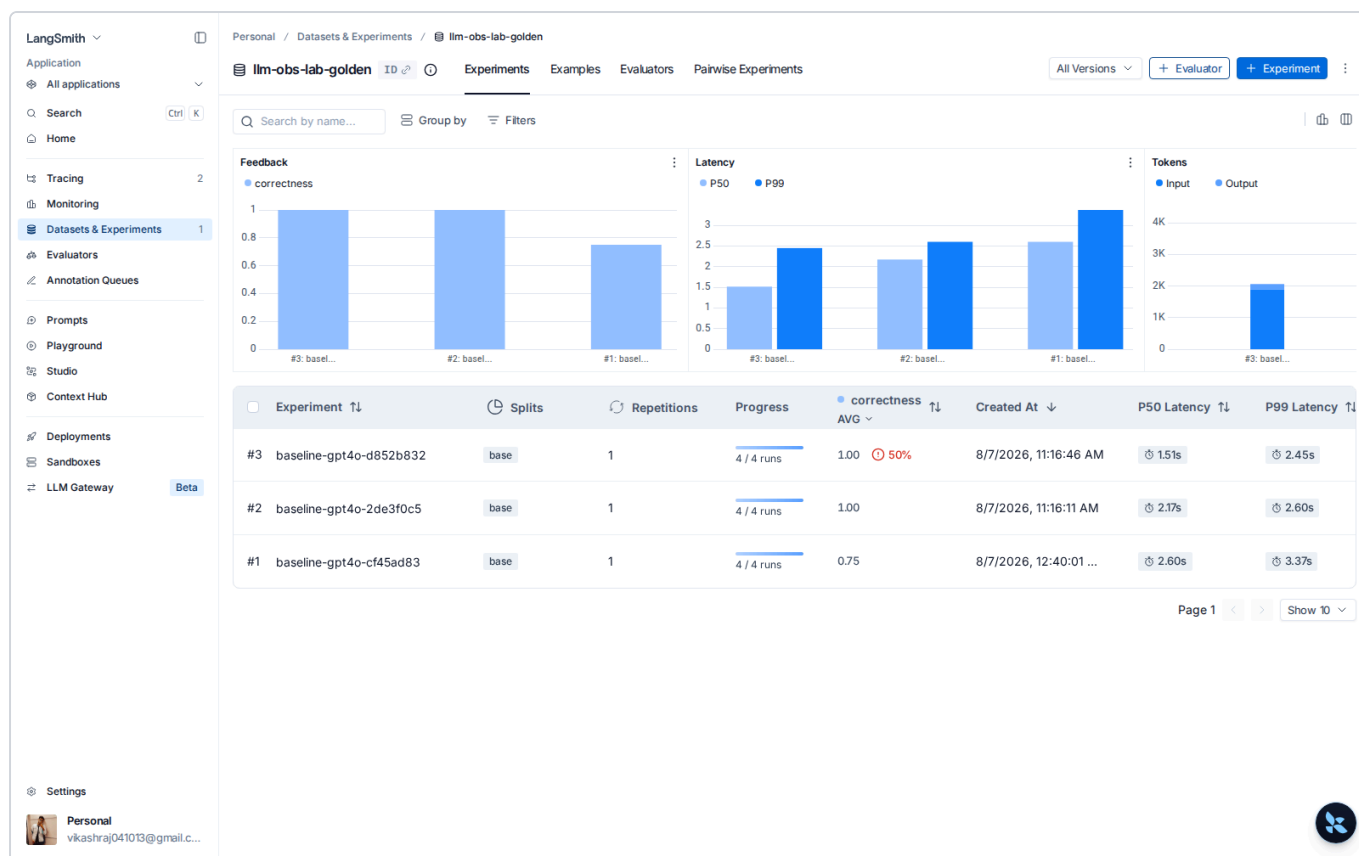
Note the second row. The agent output is `"Error: weather service timeout for city='Chennai'"` and the score is blank. **The failed case is still in the results.** It is not hidden.

And across runs:



Both runs show `Item Count 4` . The `Error Count` column (10 and 5) is how you spot a flaky agent at a glance.

LangSmith gives you the same thing with more charting on top:



Three experiments, each 4 / 4 runs, with average correctness of 1.00, 1.00 and 0.75, plus P50 and P99 latency per experiment. The red 50% badge on baseline-gpt4o-d852b832 is the forced-failure run. LangSmith surfaces error rate as a first class column next to the score, which is the fastest way to tell "my agent is wrong" apart from "my agent is broken".

The charts along the top (Feedback, Latency, Tokens) update as you add experiments, so regressions between runs are visible without opening anything.

The one real advantage Langfuse has here

Langfuse evaluators return an object with a comment field, so the judge explains itself:

```
return Evaluation(name="correctness", value=0.0, comment="Did not give the temperature.")
```

LangSmith evaluators return a bare bool. A 0.0 tells you it failed but not why. You can work around it, but Langfuse gives it to you by default.

Real numbers from the evaluation

Pulled from the LangSmith API, not typed by hand.

Clean run, everything succeeded:

Question	Score	Cost
weather in Mumbai, then multiply the temperature by 2	0.0	\$0.00278
what do my docs say about retries?	1.0	\$0.00290
what's 23*7?	1.0	\$0.00168
what's the weather in Chennai and what's 100/7?	1.0	\$0.00230

3 out of 4. The Mumbai case genuinely failed the judge. That is the eval doing its job.

Same dataset with the weather tool forced to fail every time:

Question	Result	Score
weather in Mumbai, then multiply the temperature by 2	ERROR	none
what do my docs say about retries?	ok	1.0
what's 23*7?	ok	1.0
what's the weather in Chennai and what's 100/7?	ERROR	none

Both tools behave the same way here, and this corrects a common assumption. Neither aborts the experiment. Both keep all 4 rows. Both leave the score empty on the failed ones.

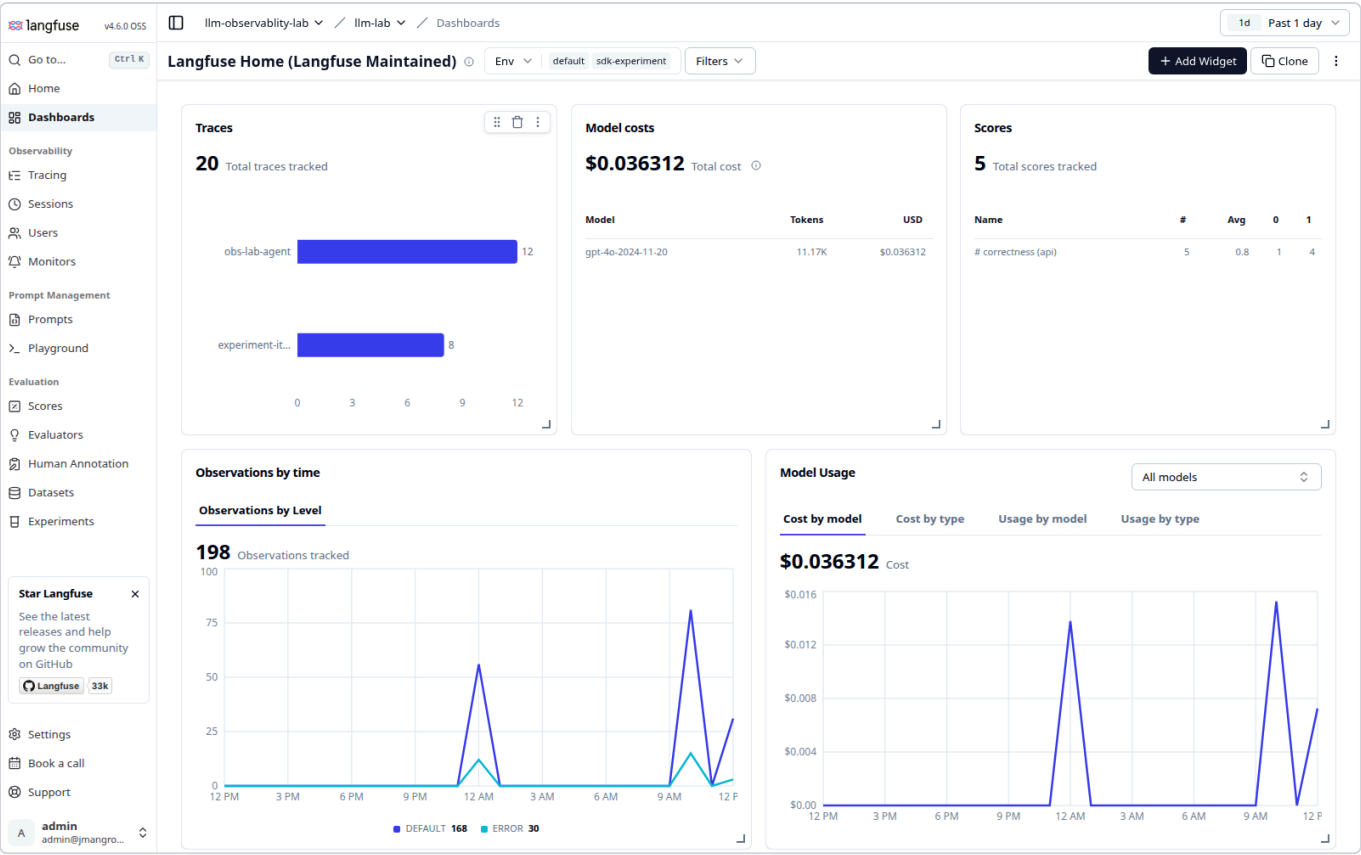
The only difference is in Python. Langfuse drops failed items from the `result.item_results` list, so the script prints `experiment finished: 2 items` even though the UI correctly shows 4. If you log that number and never look at the UI, you will think your dataset shrank. LangSmith keeps all 4 in the returned results, but then runs your evaluator on the failed row too, which throws a second error unless you guard it:

```
answer=outputs.get("answer", "") # not outputs["answer"]
```

Dashboards and feature gaps

Where the two stop being the same kind of product.

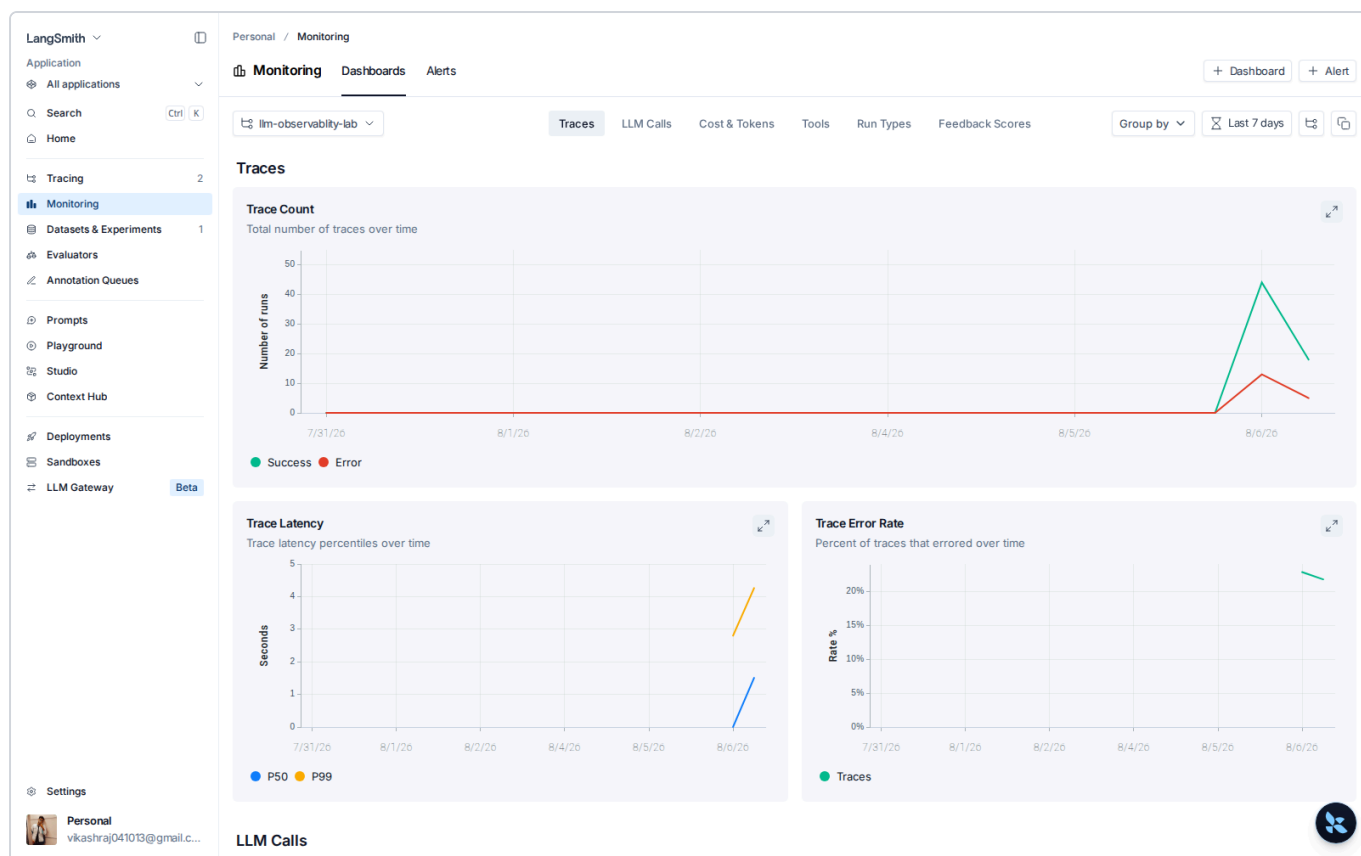
Dashboards



20 traces, \$0.036312 total, 11.17K tokens, 198 observations split DEFAULT 168 / ERROR 30 , and average score 0.8 .

That error split is the single most useful number on the page. 30 out of 198 observations errored, which is the flaky weather tool showing up as a metric instead of a surprise.

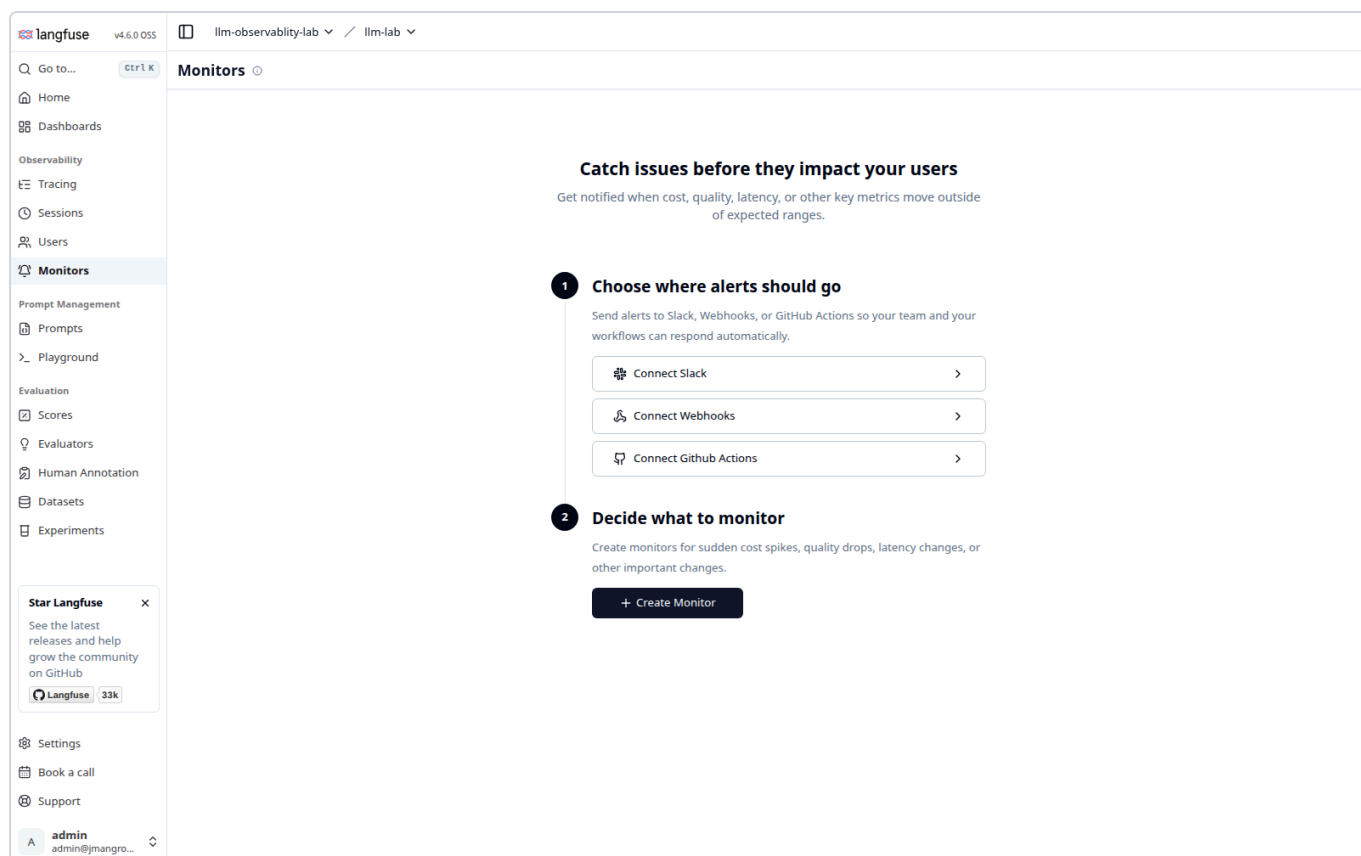
LangSmith calls this Monitoring, and it goes deeper:



Tabs for Traces, LLM Calls, Cost & Tokens, Tools, Run Types and Feedback Scores. Trace Count is split into Success and Error lines, and there is a dedicated Trace Error Rate chart.

Note the second tab at the top: **Alerts**. LangSmith can notify you when error rate or latency crosses a threshold.

Langfuse has Monitors for the same idea:



What each tool has that the other does not

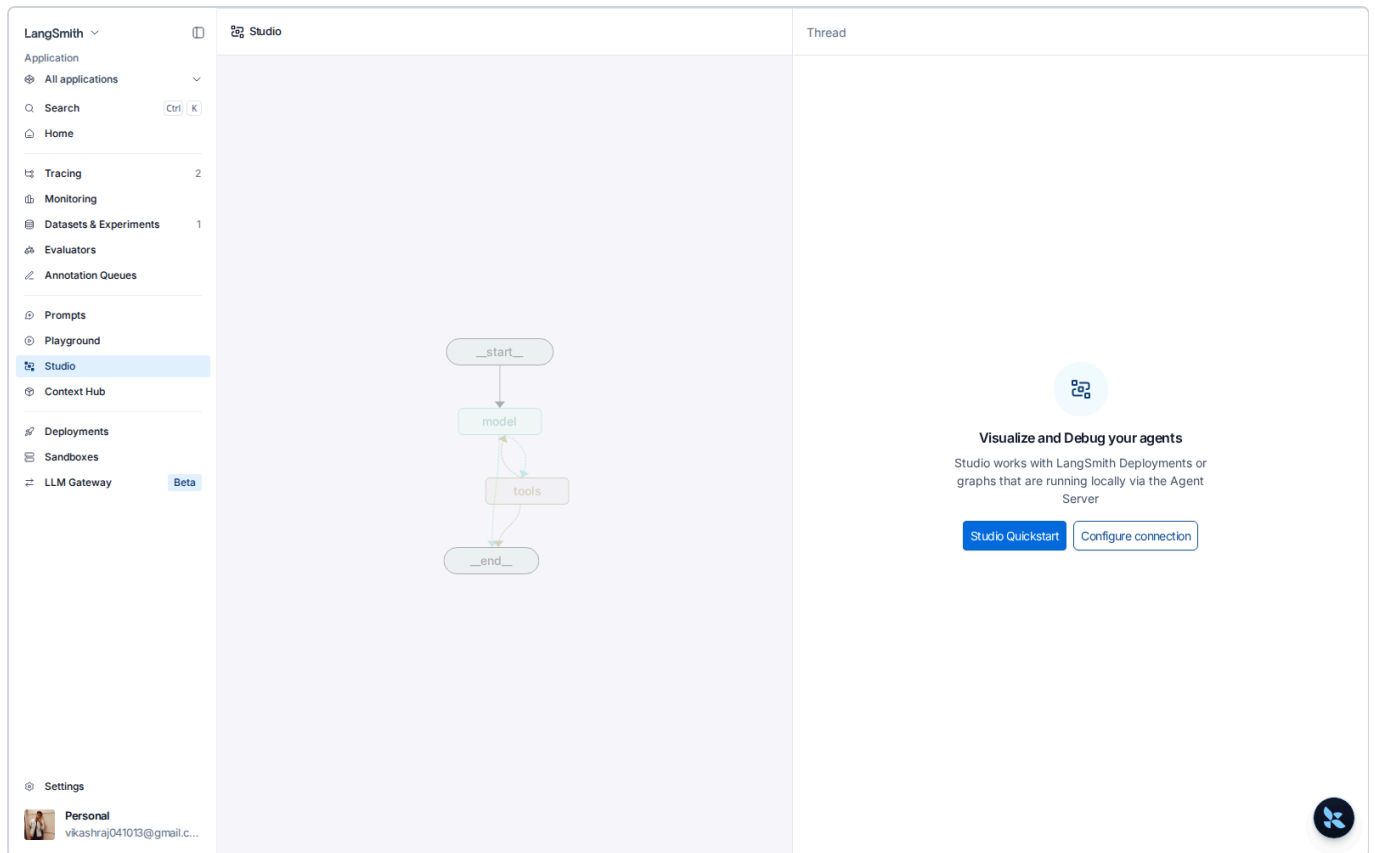
Up to here the two have been close. This is where they stop being the same kind of product.

LangSmith is turning into a platform for building and running agents. Langfuse is an observability and evaluation tool. That is the honest summary, and it cuts both ways.

Side by side, straight from the two menus:

Feature	LangSmith	Langfuse
Tracing	yes	yes
Dashboards and charts	yes (Monitoring)	yes
Alerts on thresholds	yes	yes (Monitors)
Datasets and experiments	yes	yes
LLM-as-judge evaluators	yes	yes
Human annotation	yes (Annotation Queues)	yes
Prompt management	yes	yes
Prompt playground	yes	yes
Grouping runs	Threads	Sessions
Per-user cost tracking	no dedicated page	yes
Self-hosting	no	yes
Visual agent debugger	Studio	no
Agent hosting	Deployments	no
Code sandboxes	Sandboxes	no
LLM proxy	LLM Gateway	no
Context Hub	yes	no

Studio, the one people actually mean



Studio draws your graph (`__start__` → `model` → `tools` → `__end__`) and lets you run it step by step, inspect state at each node, and edit that state before continuing. It connects to either a LangSmith Deployment or a graph running locally through the Agent Server.

This is the biggest single gap. Langfuse can show you a graph of a run that **already happened**. Studio lets you drive the graph **while it runs**, stop it mid-loop and change the state. Those are different jobs. If you build in LangGraph, Studio is a genuine reason to keep LangSmith even if your traces live elsewhere.

Deployments

LangSmith

Application

All applications

Search

Home

Tracing

Monitoring

Datasets & Experiments

Evaluators

Annotation Queues

Prompts

Playground

Studio

Context Hub

Deployments

Sandboxes

LLM Gateway

Settings

Personal

Personal / Deployments

LangSmith Deployment

Deploy and manage your AI agents.

LangSmith Studio

Upgrade required

Deploy agents fast with LangSmith Deployment

Upgrade plan to create Deployments

✓ Deploy agents with 1 click

✓ Manage agents with pre-built APIs

✓ Support long-running, bursty workloads

✓ Access to Deployment with usage-based rates

✓ 1 Dev deployment with usage included

Pricing and Plans

Access Deployments

One-click agent hosting with pre-built APIs, built for long-running and bursty workloads. Note the Upgrade required badge: this is a paid tier, not something you get on the free plan.

Sandboxes

LangSmith

Application

All applications

Search

Home

Tracing2

Monitoring

Datasets & Experiments1

Evaluators

Annotation Queues

Prompts

Playground

Studio

Context Hub

Deployments

Sandboxes

LLM GatewayBeta

Settings

Personal

vikashra041013@gmail.c...

Personal / Sandboxes

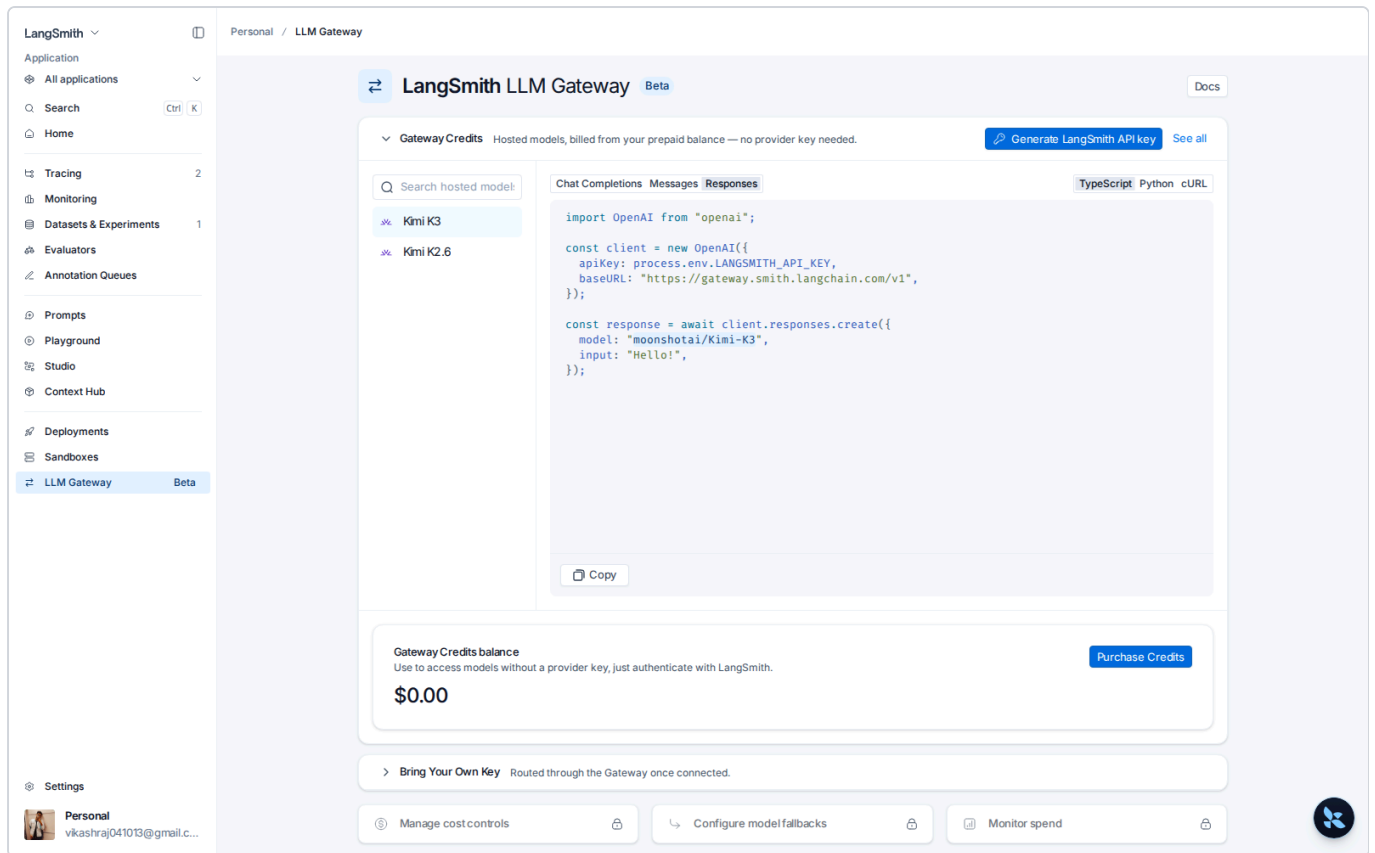
Sandboxes

Sandboxes are available on plans with sandbox access. Upgrade to unlock sandbox environments for this workspace.

Upgrade

Isolated environments for agents that execute code. Also paid.

LLM Gateway (Beta)



A proxy in front of model providers. You authenticate to LangSmith instead of holding provider keys, and it handles cost controls, model fallbacks and spend monitoring. Point your OpenAI client at `https://gateway.smith.langchain.com/v1` and it works unchanged.

For this lab that is directly relevant. The agent currently talks to Azure AI Foundry using a key in `.env`. The Gateway is the alternative to managing that key yourself.

What Langfuse has that LangSmith does not

Self-hosting is the headline and the reason this lab runs Langfuse at all. Your traces, prompts and eval results never leave your network. For anything under a data residency rule, that single fact outweighs the entire feature list above.

Per-user cost tracking gets its own page:

langfuse v4.6.0 OSS

llm-observability-lab

llm-lab

1d Past 1 day

Go to... ctrl K

Home

Dashboards

Observability

Tracing

Sessions

Users

Monitors

Prompt Management

Prompts

Playground

Evaluation

Scores

Evaluators

Human Annotation

Datasets

Experiments

Star Langfuse

See the latest releases and help grow the community on GitHub

Langfuse 33k

Settings

Book a call

Support

A

admin

admin@jmagro...

Users

Search (User ID)

Env default sdk-experiment

Filters

User ID	Environment	First Event	Last Event	Total Events	Total Tokens	Total Cost
vikash	default	8/7/2026, 1:08:53 AM	8/7/2026, 12:43:13 PM	102	6.36K	\$0.02

Rows per page 50

Page 1 of 1

<<

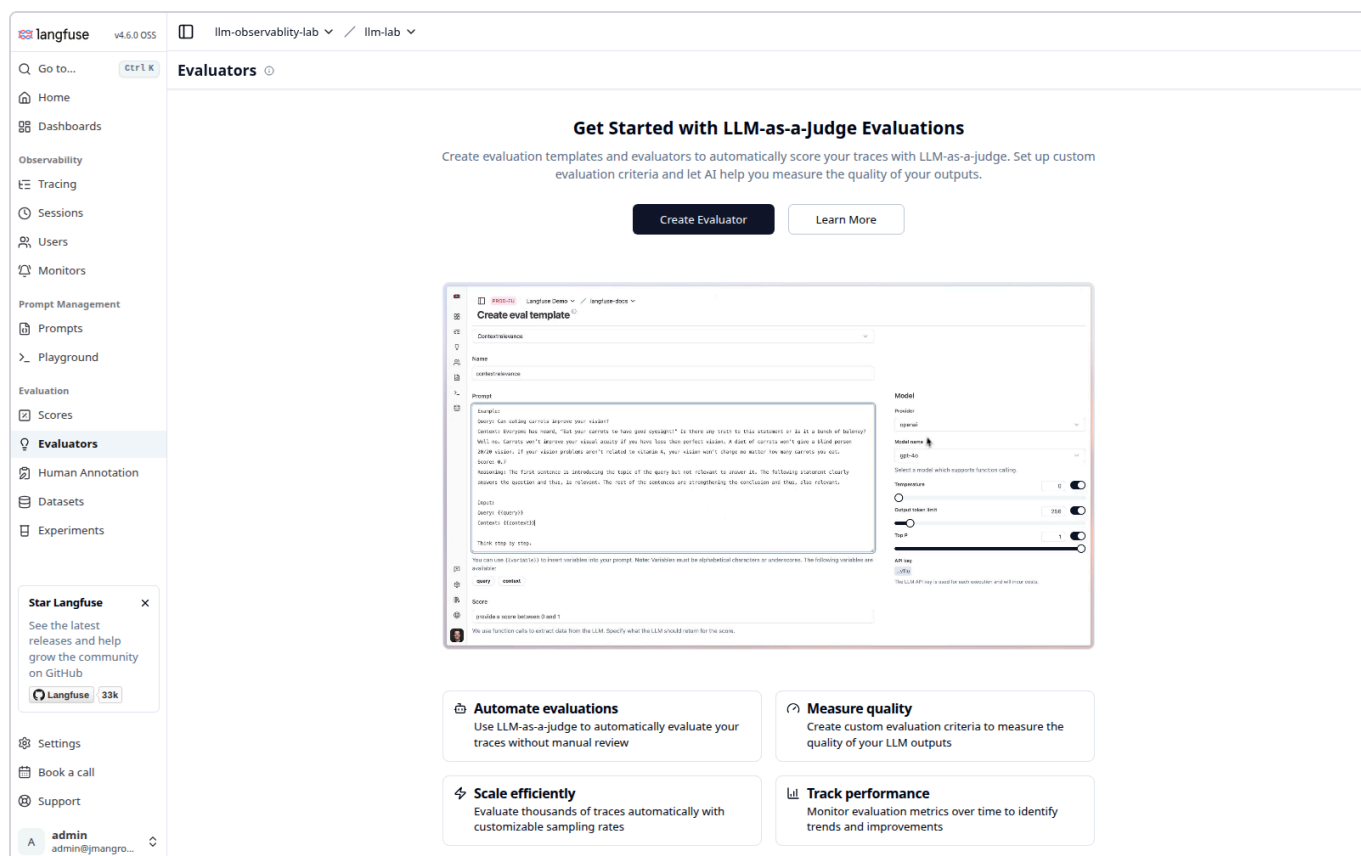
<

>

>>

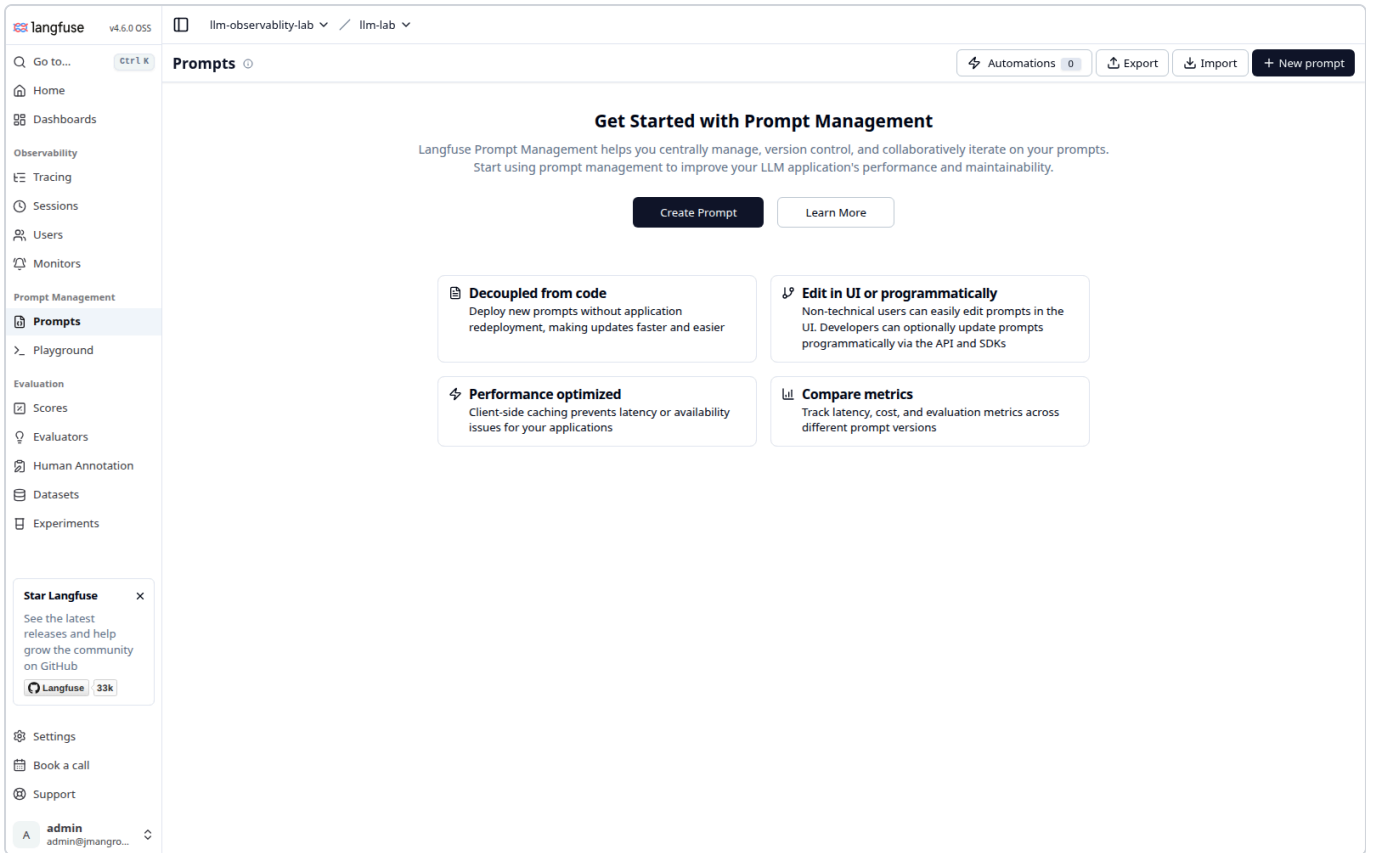
User `vikash` , 102 events, 6.36K tokens, \$0.02 spent. LangSmith stores `user_id` as ordinary metadata you can filter on. Langfuse aggregates it into a real screen. If you need to know what a specific customer costs you, Langfuse answers it in one click.

Evaluators run against live production traces, configured in the UI:

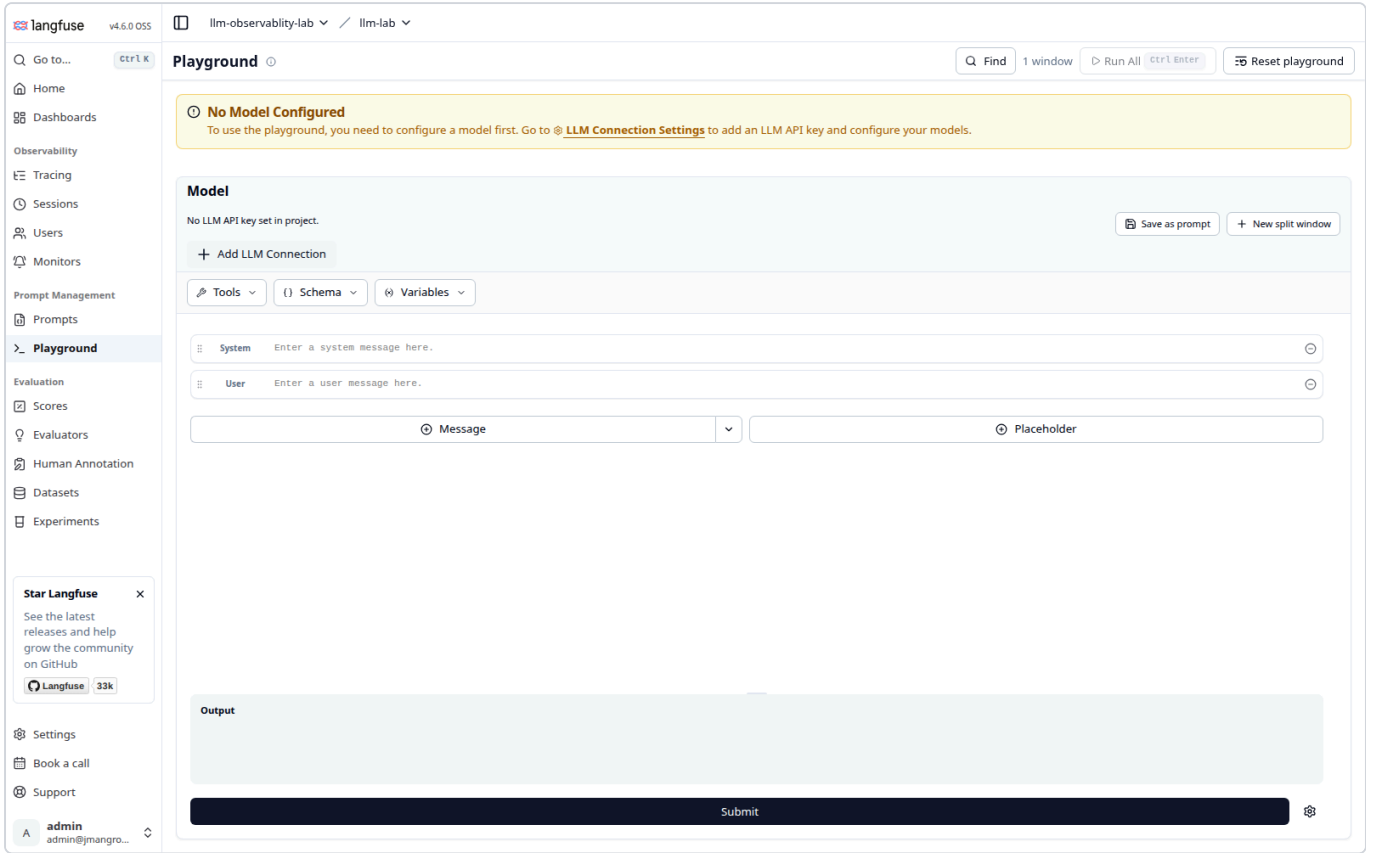


You build the judge template in a form (prompt, model, temperature, sampling rate) and it scores real traffic automatically. No code, no dataset required. LangSmith has evaluators too, but the Langfuse version is aimed squarely at continuous scoring of production traces rather than at offline dataset runs.

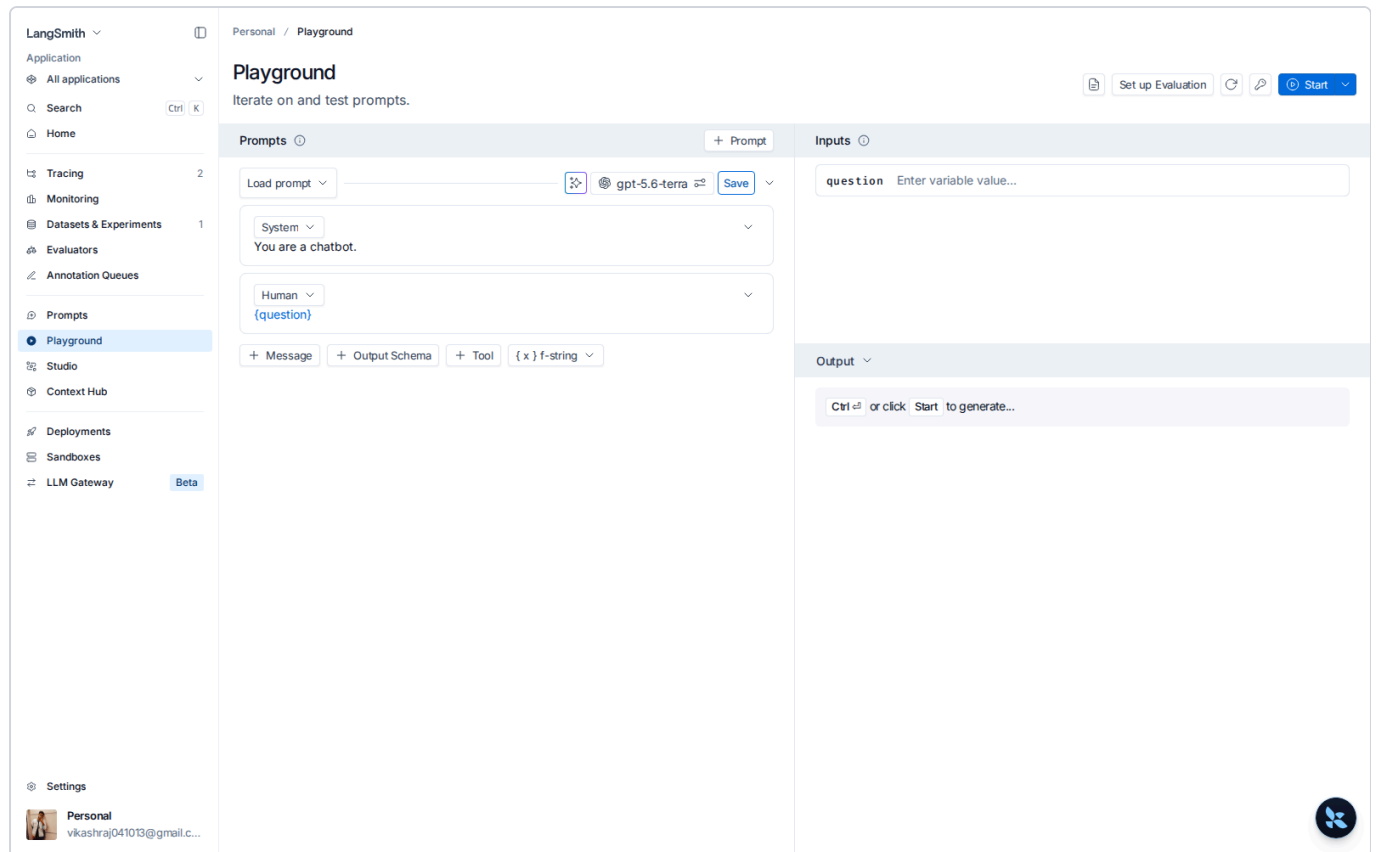
Prompt management and Playground exist on both sides, so neither wins here:



Langfuse Playground:



LangSmith Playground:



How to choose

- Building on LangGraph and you want to debug agents visually, host them, or proxy your models? **LangSmith**, and it is not close. Studio alone decides it.
- Data cannot leave your network, or you need per-user cost attribution, or you want judges scoring live traffic without writing code? **Langfuse**.
- Just want to see what your agent did and score it against a dataset? Either. They are close enough that the deciding factor is hosting, not features.

Pricing and running costs

Published prices as of August 2026, plus what the Langfuse stack actually consumes on this machine.

Pricing

Prices as published in August 2026. Check them before you commit to anything.

LangSmith

Plan	Price	Included traces	Seats	Self-hosting
Developer	\$0 then pay as you go	5k base traces / month	1 only	no
Plus	\$39 / seat / month then pay as you go	10k base traces / month	unlimited	no
Enterprise	custom	custom	custom	yes

Past the included allowance you pay per trace, and retention is what drives the price:

Trace type	Price	Retention
Base	\$0.50 per 1,000 traces (0.05 cents each)	14 days
Extended	\$5.00 per 1,000 traces (0.50 cents each)	400 days

Extended retention is **10x** the base price. Upgrading a base trace afterwards costs 0.45 cents. Traces with feedback attached (evaluator scores, human labels) get upgraded, which is exactly what your eval runs produce. Worth knowing before you run a large experiment.

Two things gated behind Enterprise: **self-hosting** and **custom SSO**. Deployments and Sandboxes are paid on top of the plan. On the Developer plan you get 5 LCU and 1 LSU of sandbox usage a month, capped at 10 sandboxes.

Langfuse

Self-hosted:

Tier	Price	What you get
Open Source	Free, MIT licence	Everything in this document. Unlimited projects, users, traces.
Self-hosted Enterprise	custom	RBAC, audit logs, data masking, retention policies, SOC 2 / ISO 27001 docs, SLA

Cloud:

Plan	Price	Included units	Extra units	Retention
Hobby	\$0	50k / month	none	30 days
Core	\$29 / month	100k / month	\$8 per 100k	90 days
Pro	\$199 / month	100k / month	\$8 per 100k	3 years
Enterprise	\$2,499 / month	100k / month	\$8 per 100k	3 years

The trap: units are not traces

LangSmith bills per **trace**. Langfuse bills per **unit**, which is roughly one observation (one span).

This lab's own dashboard settles it: **20 traces produced 198 observations**, so this agent averages **9.9 units per trace**.

That changes the free tiers completely:

	Free allowance	In traces, for this agent
LangSmith Developer	5k traces	5,000 traces / month
Langfuse Cloud Hobby	50k units	~5,050 traces / month
Langfuse Cloud Core (\$29)	100k units	~10,100 traces / month

The free tiers land within one percent of each other. A per-unit price that looks ten times cheaper is not, once your agent has a few tool calls in the loop. A chattier agent makes this worse.

What Langfuse actually costs to run

Measured on this machine (14 cores, 15 GiB RAM) with the stack idle after the lab runs.

Memory and CPU, per container

Container	Memory	CPU
langfuse-web	888.3 MiB	0.24%
clickhouse	669.4 MiB	4.61%
langfuse-worker	533.0 MiB	0.26%
minio	78.5 MiB	3.64%
postgres	47.1 MiB	0.00%
redis	6.4 MiB	2.06%
Total	2.16 GiB	~11% of one core

Disk

What	Size
Docker images (6)	~6.0 GB
Git clone	261 MB
ClickHouse data + logs	360 MB
Postgres data	46 MB
MinIO + Redis data	< 1 MB
Total after 20 traces	~6.7 GB

Note the shape of that. Almost all of it is **images, not data**. `langfuse-worker` alone is 2.07 GB and `langfuse-web` is 1.75 GB. You pay 6 GB of disk before recording a single trace.

Measured versus recommended

Langfuse officially recommends **4 cores, 16 GiB RAM, and 100 GiB storage** (a `t3.xlarge` class VM).

The gap is large: 2.16 GiB measured against 16 GiB recommended. Both numbers are honest. The recommendation covers production ingest, where ClickHouse is absorbing continuous writes rather than sitting idle. For a lab or a small team, the real figure is closer to what is measured here.

The practical read on a 16 GiB laptop:

- Langfuse takes about **14%** of total RAM at idle
- That is fine alongside an IDE and a browser, but it is not free
- **ClickHouse is the reason this is six containers and not one binary**. It is the largest memory user, the largest disk user, and the busiest at idle. It is also what makes trace queries fast at volume.

The docs are explicit that docker compose is for trying Langfuse out, not for running it. It has **no high availability, no horizontal scaling, and no backup**. Production means Kubernetes.

Operational friction, measured in this lab

	LangSmith	Langfuse
Setup	Nothing to run	6 containers, ~6.7 GB
RAM	0	2.16 GiB idle, 16 GiB recommended
Ports claimed	none	3000, 3030, 5432, 8123, 9000, 9090, 9091, 6379
Your data	leaves your network	stays local
Ongoing work	none	port clashes, container health, backups are your problem

Port `5432` is the one that actually bit. Any other local Postgres stops the whole stack from starting, and Compose aborts the entire `up` when one container fails to bind.

When does self-hosting actually save money

The licence is free. The machine is not.

A 4 core / 16 GiB VM plus 100 GiB of storage is the floor, and at typical cloud on-demand rates that lands somewhere around **\$100 to \$150 a month**. That figure is an estimate from published instance pricing, not something measured here.

Compare that against Langfuse Cloud Core at **\$29 a month** for 100k units, plus **\$8 per additional 100k**:

```
self-hosted VM  ~= $130 / month, flat
Langfuse Cloud  =  $29 + $8 per extra 100k units
break-even      ~=  1.3 million units / month
                ~=  130,000 traces / month for this agent
```

Below roughly 130k traces a month, self-hosting costs more, not less. You are paying for a VM that is mostly idle.

So the reason to self-host is almost never the bill. It is that the data cannot leave your network. If that constraint is real, the cost question does not arise. If it is not real, cloud is cheaper until you are at serious volume.

One more thing the price lists do not show: **someone has to run it**. Upgrades, backups, disk filling with ClickHouse data, the 3am page when the worker dies. That is not on any pricing page and it usually costs more than the VM.

How much can self-hosted Langfuse actually handle

Langfuse publishes no throughput numbers. The self-hosting docs say "at least 2 CPUs and 4 GB of RAM per container" and "add instances once CPU exceeds 50%", and stop there.

So this was measured directly, on the Docker Compose stack in this lab.

Short answer: 395 traces per second sustained, and roughly 14 million traces per 100 GB of disk. For almost any real application, that is far more headroom than you will need, and disk will run out long before throughput does.

How it was measured

A synthetic load generator pushing traces shaped like the real agent: one trace containing about 10 observations (agent spans, two generations with token counts, `should_continue`, a tools span, and two tool spans).

No LLM calls anywhere in the loop, so this measures **Langfuse ingest**, not model latency.

Two numbers were tracked separately, because they are very different things:

1. **Client-side push**, how fast the SDK accepts and queues events
2. **End-to-end**, how long until the rows are actually queryable in ClickHouse

The second is the honest number. `langfuse.flush()` returns in 0.0 seconds because the SDK hands off to a background queue. If you only measure the flush, you will conclude Langfuse is infinitely fast, which is wrong. Server-side confirmation came from counting rows directly:

```
docker exec langfuse-clickhouse-1 clickhouse-client \  
  --query "SELECT uniqExact(trace_id), count() FROM default.events_core"
```

Test machine: **14 cores, 15 GiB RAM**, all six containers on one host, loopback network.

Throughput

6,000 traces and 60,199 events pushed in total.

Measurement	Result
Client push rate	499 traces/s
End-to-end, confirmed in ClickHouse	395 traces/s
Events per second, end-to-end	3,952 events/s
50,000 events ingested in	12.7 seconds

The worker keeps up with the client almost exactly. There was no growing backlog and no dropped events, so at this scale ingest is not the bottleneck.

Put in terms you can act on:

Your workload	Traces/second	Percent of measured capacity
10,000 traces / day	0.12	0.03%
100,000 traces / day	1.2	0.3%
1,000,000 traces / day	11.6	2.9%
34,000,000 traces / day	395	100%

A busy production agent doing 100k traces a day uses **less than half a percent** of what this single-node stack handled.

What that load costs in CPU and memory

Sampled once per second across the whole run.

Container	Idle memory	Peak memory	Peak CPU
clickhouse	557 MiB	1,000 MiB	91.5%
langfuse-web	840 MiB	937 MiB	96.5%
langfuse-worker	535 MiB	966 MiB	75.4%
minio	85 MiB	159 MiB	9.0%
postgres	48 MiB	116 MiB	4.8%
redis	9 MiB	10 MiB	3.5%
Total	2.16 GiB	~3.2 GiB	

Memory grows by about **1 GiB** from idle to full load. That is modest, and it is why the official 16 GiB recommendation is generous for anything short of sustained production ingest.

CPU is the real constraint. Three containers each hit 75 to 96 percent of a core at maximum push. Langfuse's own guidance is to add instances past 50% CPU, so **395 traces/s is the ceiling for this single node, not a comfortable operating point**. Plan for roughly half that as sustainable: call it **200 traces/s**, or 17 million a day.

Storage, which is what actually runs out

ClickHouse compresses well. Measured from `system.parts` :

Table	On disk	Uncompressed	Ratio	Rows
events_core	4.58 MiB	64.41 MiB	14.1x	60,199
events_full	4.64 MiB	64.66 MiB	13.9x	60,199

But ClickHouse is not the whole story. Large payloads go to MinIO as blobs, and that turned out to be **the bigger consumer**:

Component	Growth over 6,020 traces
ClickHouse on disk	9.67 MB
MinIO blob storage	34.12 MB
Postgres	0 MB (metadata only, does not grow with traces)
Total	43.79 MB

Which gives the number worth writing down:

7.27 MB per 1,000 traces
7.1 GB per 1,000,000 traces

Disk	Holds roughly
100 GB (the Langfuse recommendation)	14.1 million traces
500 GB	70.4 million traces
1 TB	140.8 million traces

At 1 million traces a month you would use about **7 GB a month**, so a 100 GB disk lasts over a year before you need to think about retention.

Two things to note. Postgres does not grow with trace volume, it holds users, projects and datasets, so do not size it for traffic. And this figure scales with **payload size**: this agent has small inputs and outputs, and an agent stuffing large RAG contexts into its prompts will push far more into MinIO per trace.

Caveats, because the number is not the whole truth

Being straight about what this test does not prove:

- **Ingest only.** Query performance at 100 million rows was not tested. That is usually where ClickHouse earns its keep, but it is untested here.
 - **Loopback network.** No real network latency between client and server. A remote client will be slower.
 - **Single node, no HA.** Langfuse's docs are explicit that Docker Compose "lacks high-availability, scaling capabilities, and backup functionality". Production means Kubernetes.
 - **Burst, not soak.** This ran for seconds, not days. It says nothing about ClickHouse merge pressure or disk fragmentation over months.
 - **Nothing else on the box.** All 14 cores were available.
-

Versus LangSmith

The two have completely different failure modes at volume, and that is the whole point.

	Self-hosted Langfuse	LangSmith Cloud
Throughput ceiling	395 traces/s measured , ~200/s sustainable per node	Their problem, not yours
Documented rate limit	none	6,000 requests / 10 seconds on /runs/multipart
What happens at the limit	ingest lag, then dropped events	HTTP 429, you back off
How you get more	add nodes, move to Kubernetes	pay more
Storage limit	your disk, 14M traces per 100 GB	none, but retention is 14 days by default
What actually stops you	disk and ops effort	the invoice

That last row matters more than any throughput figure. **On LangSmith you cannot run out of capacity, only out of budget. On Langfuse you cannot run out of budget, only out of disk and patience.**

What volume costs on each

Using the measured 10 observations per trace, and the published prices:

Traces / month	LangSmith (base, \$0.50/1k)	Langfuse Cloud (\$29 + \$8/100k units)	Self-hosted Langfuse
10k	\$2.50	\$29	~\$130 VM
100k	\$47.50	~\$101	~\$130 VM
1M	\$497.50	\$821	~\$130 VM + 7 GB disk
10M	\$4,997.50	\$7,949	~\$130 VM + 71 GB disk

Self-hosting is a **flat cost**. Both hosted options scale linearly with volume. The crossover sits somewhere around 130k traces a month, and past 1 million the gap becomes the entire argument.

The retention trap

LangSmith base traces are kept **14 days**. Keeping them 400 days costs **10x**, and traces with feedback attached get upgraded automatically, which is exactly what your evaluation runs produce.

Self-hosted Langfuse keeps everything until you delete it. At 7.27 MB per 1,000 traces, a year of 1M traces a month is about **85 GB**. That is one cheap disk against a 10x retention multiplier.

Sizing recommendation

Your volume	What to run
Under 100k traces / month	LangSmith or Langfuse Cloud . Self-hosting costs more than it saves.
100k to 1M / month	Either. Decide on data residency, not on price.
Over 1M / month	Self-hosted Langfuse , if you have someone to run it.
Over 17M / day	Kubernetes, multiple web and worker replicas. One node will not do it.

And regardless of volume: if the data cannot leave your network, none of the above applies. Self-host and size the disk.

Reproducing this

The load generator is `scripts/loadtest.py`. It needs a working `.env` with the `LANGFUSE_*` variables.

```
uv run python scripts/loadtest.py 5000    # 5,000 traces, ~50,000 observations
```

Then confirm server-side, which is the number that counts:

```
docker exec langfuse-clickhouse-1 clickhouse-client \
  --query "SELECT uniqExact(trace_id) AS traces, count() AS events FROM default.events_core"
```

[!WARNING] This writes real traces into your Langfuse project. This lab's instance now holds about 6,000 synthetic `obs-lab-agent` traces from the benchmark, mixed in with the genuine ones. Use a throwaway project if that matters to you.
